

Unit 4

Basics of Inheritance:

Inheritance

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

Terms used in Inheritance

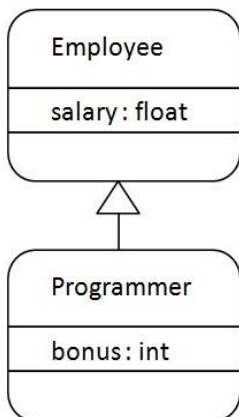
- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

1. **class** Subclass-name **extends** Superclass-name
2. {
3. //methods and fields
4. }

The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

Java Inheritance Example



As displayed in the above figure, Programmer is the subclass and Employee is the superclass. The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.

```
class Employee{
    float salary=40000;
}
class Programmer extends Employee{
    int bonus=10000;
```

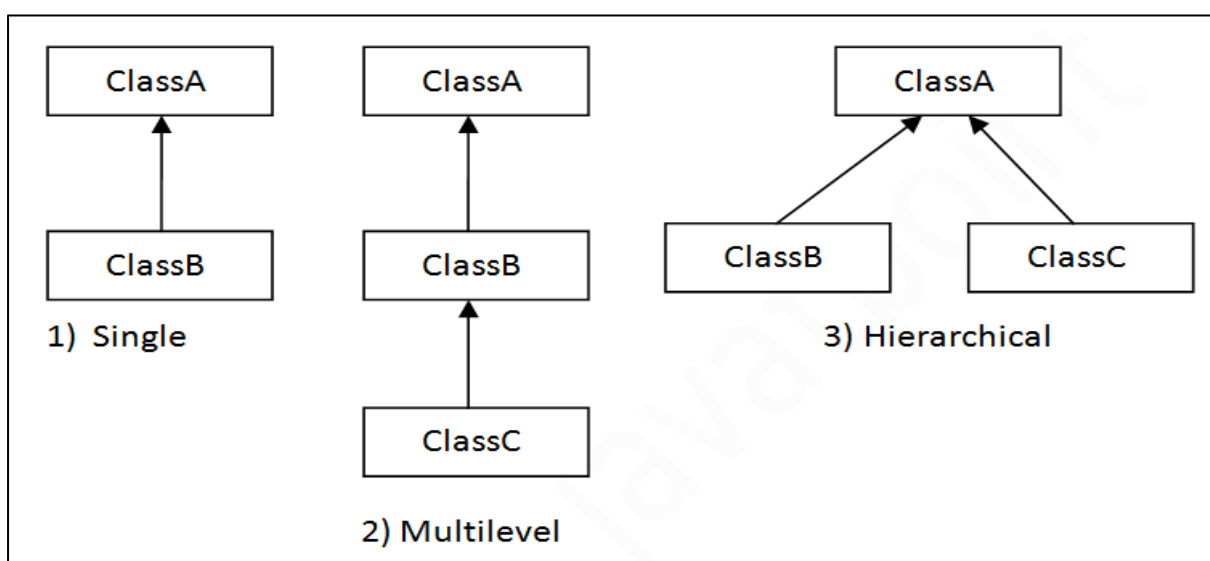
```
public static void main(String args[]){
    Programmer p=new Programmer();
    System.out.println("Programmer salary is:"+p.salary);
    System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

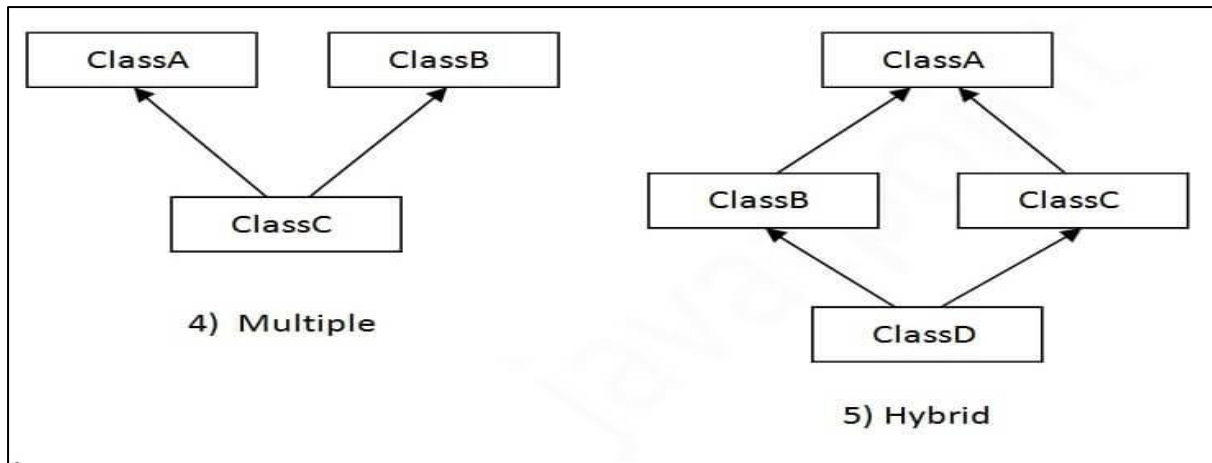
Programmer salary is:40000.0

Bonus of programmer is:10000

In the above example, Programmer object can access the field of own class as well as of Employee class i.e. code reusability.

Types of inheritance:





Note: Multiple inheritance is not supported in Java through class.

When one class inherits multiple classes, it is known as multiple inheritance.

Single inheritance On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

Single Inheritance Example

When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

File: TestInheritance.java

```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class TestInheritance{
    public static void main(String args[]){
        Dog d=new Dog();
        d.bark();
        d.eat();
    }
}
```

Output:

barking...

eating...

Multiple inheritance**Multiple Inheritance (Not Supported Using Classes)***Definition*

Multiple inheritance is when a class inherits from **more than one parent class**. Java **does not support multiple inheritance using classes** to avoid ambiguity.

Q) Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class. Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

```
class A{
    void msg()
    {
        System.out.println("Hello");
    }
}
class B
{
    void msg(){
        System.out.println("Welcome");
    }
}
class C extends A,B{//suppose if it were

public static void main(String args[]){
    C obj=new C();
    obj.msg();//Now which msg() method would be invoked?
```

```
}  
}
```

Multilevel inheritance

When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

File: TestInheritance2.java

```
class Animal{  
    void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
    void bark(){System.out.println("barking...");}  
}  
class BabyDog extends Dog {  
    void weep(){System.out.println("weeping...");}  
}  
class TestInheritance2{  
    public static void main(String args[]){  
        BabyDog d=new BabyDog();  
        d.weep();  
        d.bark();  
        d.eat();  
    }  
}
```

Output:

```
weeping...  
barking...  
eating...
```

Hierarchical Inheritance

When two or more classes inherits a single class, it is known as *hierarchical inheritance*. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

File: TestInheritance3.java

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}
}
```

Output:
meowing...

Hybrid inheritance

Definition

Hybrid inheritance is a **combination of two or more types of inheritance**. In Java, it is achieved **using interfaces**, not classes.

concepts of method overriding

Definition

Method overriding occurs when a **subclass provides a specific implementation of a method already defined in its superclass** using the same method signature.

Syntax

```
class Child extends Parent {
void methodName() {
// new implementation
}
```

```
}
```

Example Code

```
class Bank {  
    void rate() {  
        System.out.println("Base rate");  
    }  
}  
  
class SBI extends Bank {  
    void rate() {  
        System.out.println("SBI rate is 7%");  
    }  
}
```

Output

SBI rate is 7%

Code Explanation (3 Lines)

- rate() method is defined in both parent and child class.
- Child class method overrides the parent class method.
- Method selection happens at runtime (runtime polymorphism).

extending class

The extends keyword is used to **create a subclass from a superclass**.

Example

```
class Parent {  
    int x = 10;  
}  
  
class Child extends Parent {  
    void show() {  
        System.out.println(x);  
    }  
}
```

super class

A **superclass** is the class whose properties are inherited by another class. The super keyword is used to **refer to superclass members**.

Example

```
class Parent {
    int a = 50;
}

class Child extends Parent {
    void display() {
        System.out.println(super.a);
    }
}
```

Output

50

Explanation

- super.a accesses the variable of the parent class.
- It avoids confusion when parent and child have same variable names.

Abstract Class

Definition

An abstract class is a class declared using the abstract keyword. It **cannot be instantiated** and may contain **abstract methods (without body)** and **non-abstract methods**.

Syntax

```
abstract class ClassName {
    abstract void method();
}
```

Example Code

```
abstract class Vehicle {
    abstract void start();
}

class Bike extends Vehicle {
    void start() {
        System.out.println("Bike starts");
    }
}
```



```

}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Bike();
        v.start();
    }
}

```

Output

Bike starts

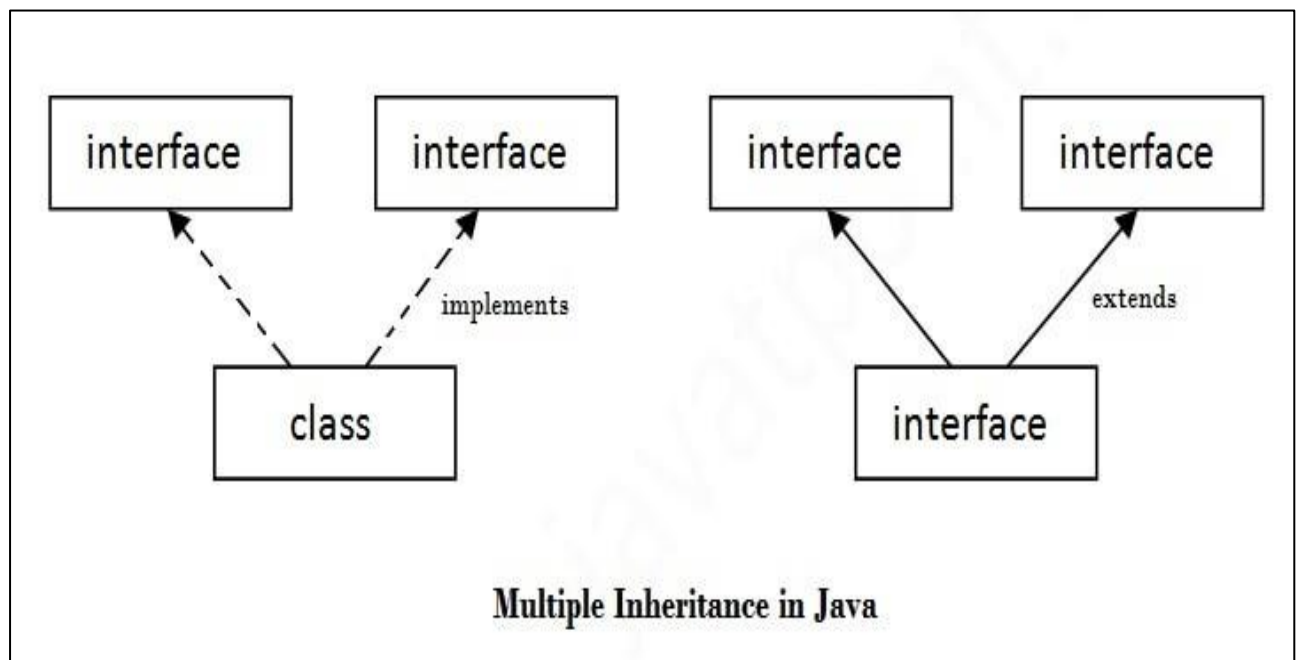
Explanation

- Abstract class defines rules but not complete implementation.
- Subclass must implement all abstract methods.
- Object is created using subclass reference.

multiple inheritance using interface

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



```

interface Printable{
    void print();
}
interface Showable{

```

```
void show();  
}  
class A7 implements Printable, Showable{  
public void print(){System.out.println("Hello");}  
public void show(){System.out.println("Welcome");}
```

```
public static void main(String args[]){  
    A7 obj = new A7();  
    obj.print();  
    obj.show();  
}  
}
```

Output: Hello
 Welcome

Interface inheritance

A class implements an interface, but one interface extends another interface.

```
interface Printable{  
void print();  
}  
interface Showable extends Printable{  
void show();  
}  
class TestInterface4 implements Showable{  
public void print(){System.out.println("Hello");}  
public void show(){System.out.println("Welcome");}
```

```
public static void main(String args[]){  
    TestInterface4 obj = new TestInterface4();  
    obj.print();  
    obj.show();  
}  
}
```

Output:

Hello
Welcome